An

Industrial Training Report

On

Python Basics

Submitted partial fulfillment for the award of

degree of

Bachelor of Technology

In

Computer Science and Engineering

Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT/CYBER SECURITY Dept.

Submitted By:

Mayank Phalodia

24EGJCS123

Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Session: 2025-26

Certificate

Acknowledgement

I express my sincere gratitude to **Mr. AshaRam Gurjar**, Assistant Professor, Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for providing me the opportunity to undergo Industrial Training in the domain of **Python Programming**. His valuable guidance, continuous support, and insightful feedback greatly enhanced my understanding of Python fundamentals and real-world applications of the language.

I extend my heartfelt thanks to the entire team of **GIT, Jaipur** for their support and encouragement throughout the training period. Their expertise, practical demonstrations, and concept-based mentoring helped me gain hands-on experience with essential Python topics such as data types, control statements, functions, modules, file handling, exception handling, and basic project development.

I am also thankful to **Dr. I. C. Sharma**, Principal of GIT, and **Dr. Pradeep Jha**, Head of the Department of Computer Science and Engineering, for their motivation and academic guidance, which inspired me to successfully complete this industrial training.

Lastly, I express my deep gratitude to my parents, friends, and fellow learners for their constant encouragement, cooperation, and moral support throughout the entire training duration.

Place: GIT , Jaipur
Mayank Phalodia
24EGJCS123
B.Tech III Semester, II Year, CSE

Abstract

This industrial training focused on developing a strong foundation in **Python programming** through a practical and concept-oriented learning approach. The program began with essential Python fundamentals such as variables, data types, operators, input/output handling, and progressed toward core programming concepts including control statements, functions, modules, and file handling. I gained hands-on experience in writing clean, readable Python code, implementing logical problem-solving techniques, and understanding Python's dynamic typing and built-in libraries.

The training also covered important topics such as data structures (lists, tuples, sets, dictionaries), string manipulation, exception handling, and working with external modules. Through small tasks and mini-projects, I learned how to structure Python programs efficiently, handle errors gracefully, and apply modular programming practices for real-world scenarios.

Overall, this training strengthened my programming logic and provided me with the technical proficiency needed to build basic Python applications while preparing a solid foundation for advanced domains like data analysis, automation, and web development.

Table of Contents

List of Figures

List of Tables

List of Symbols

- **IDE** — Integrated Development Environment
- **CLI** — Command Line Interface
- **PVM** — Python Virtual Machine
- **PEP** — Python Enhancement Proposal
- **I/O** — Input / Output
- **API** — Application Programming Interface
- **OOP** — Object-Oriented Programming
- **CSV** — Comma-Separated Values (file format)
- **JSON** — JavaScript Object Notation (data format)
- **pip** — Python Package Installer
- **.py** — Python Source File Extension
- **REPL** — Read-Eval-Print Loop (Python Shell)
- **RAM** — Random Access Memory
- **CPU** — Central Processing Unit

Chapter 1

Introduction to Python Programming

Today marked the beginning of my industrial training in **Python Programming**, where I was introduced to the fundamental concepts, applications, and importance of Python in the modern technology landscape. The session focused on understanding what Python is, why it is widely used, and how it serves as a beginner-friendly yet powerful programming language.

What is Python?

Python is a high-level, interpreted, and general-purpose programming language developed by **Guido van Rossum** and first released in 1991. It is known for its **simple syntax**, **readability**, and **versatile applications**, making it one of the most popular languages for students, developers, and industries worldwide.

Python emphasizes code clarity, allowing beginners to learn programming concepts quickly without worrying about complex syntax.

Why Python is Popular?

During the training, I learned about several reasons why Python is preferred across domains:

- **Easy to read and write:** Uses simple English-like syntax
- **Beginner-friendly:** Ideal for learning logic and programming basics
- **Platform independent:** Runs on Windows, Linux, and macOS
- **Large standard library:** Provides built-in modules for many tasks
- **Versatile usage:** Used in development, automation, data science, AI, ML, IoT, and more
- **Huge community support:** Many online resources and packages available

Applications of Python

My trainer explained various real-world uses where Python plays an important role:

- **Web Development (Django, Flask)**
- **Data Science & Machine Learning (NumPy, pandas, scikit-learn)**
- **Automation and Scripting**
- **Cybersecurity tools and scanners**
- **IoT and Robotics**
- **Game Development (Pygame)**
- **Desktop Applications (Tkinter, PyQt)**

This gave me an overview of how Python is used from simple scripting to advanced technologies.

Python Installation & Setup

On the first day, I learned how to install Python and set up a development environment.

Steps Included:

1. Downloaded **Python 3.x** from the official website
2. Installed it with *Add to PATH* enabled
3. Verified installation using the terminal (python --version)
4. Installed **VS Code** as the primary editor
5. Explored the Python extension in VS Code
6. Ran the first program:

```
print("Hello, Python!")
```

Basic Python Architecture Overview

Python follows a simple execution process:

Python source code → .py file

Code is converted into **bytecode**

Executed by the **Python Virtual Machine (PVM)**

Output is shown to the user

This structure helps Python remain portable and consistent across platforms.

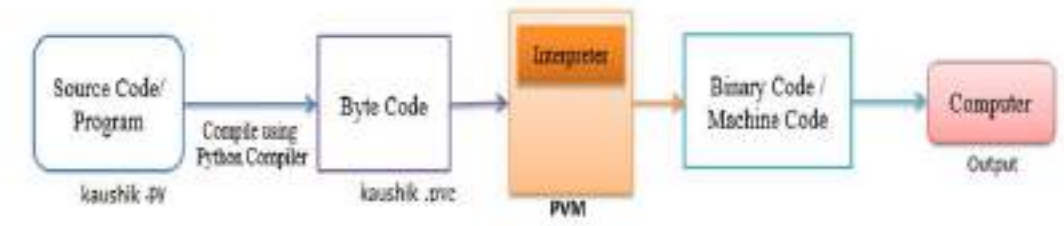


Figure 01: Python Execution Flow (Source Code → Bytecode → PVM → Output)

Key Characteristics of Python

- Interpreted language (executes line by line)
- Dynamically typed (no need to declare variable type)
- Supports multiple programming paradigms
- Comes with built-in functions and libraries
- Easy to integrate with databases and other languages

Chapter 2

Python Syntax, Variables & Data Types

Today's session introduced me to the fundamental building blocks of Python programming—its syntax rules, the use of variables, and the different types of data supported by the language. Since Python is known for its readability, simplicity, and beginner-friendly nature, understanding these basics became a solid foundation for all the concepts I learned afterward. The trainer emphasized that even the most advanced Python applications—whether in AI, automation, API development, or data science—are built upon these core principles.

Understanding Python Syntax

Python follows a clean and simple syntax compared to many other programming languages like C or Java. One of the major highlights of Python syntax is that it uses **indentation** instead of braces {} to define code blocks. This promotes clean coding habits and forces developers to write readable code.

Key Characteristics of Python Syntax

a) Indentation

Python uses spaces or tabs to define blocks of code.
Example:

```
if score > 50:

    print("Pass")else:

    print("Fail")
```

If indentation is not correct, Python throws an IndentationError.
This concept was new initially, but after practicing small programs, it became intuitive.

b) Case Sensitivity

Python is case-sensitive:

Name = "Mayank"

name = "Phalodia"

Both are treated as different variables.

c) Comments

Comments are used to explain code logic or temporarily disable code.

Single-line comment:

```
# This is a comment
```

Multi-line comment:

```
"""
This is a
multi-line comment
"""
```

I learned that adding comments is extremely important to improve code readability, especially when working on actual industrial projects.

d) Print Statement

The print() function is one of the most frequently used statements in Python.

```
print("Hello, Python")
```

It helped me verify outputs, test variable values, and debug simple programs during the training.

Variables in Python

Variables store values in memory so they can be used later in the program. During the training, I learned that Python does not require specifying a data type while creating a variable. The interpreter automatically assigns the type based on the value.

Example:

```
x = 10          # Integer

name = "GIT"    # String

isValid = True  # Boolean
```

This feature is called **dynamic typing**, and it makes Python very flexible.

Variable Naming Rules

The trainer explained the following rules for naming variables:

- A variable name **must start** with a letter or underscore
- Cannot start with a number
- Cannot contain special characters like @, %, $, !
- Python keywords like for, while, class, break cannot be used as variable names

Examples:

Valid:

```
rollNo, student_name, _value
```

Invalid:

```
2name, student-name, class
```

Variable Assignment

Python allows assigning the same value to multiple variables:

```
a = b = c = 50
```

It also allows assigning multiple values at once:

```
x, y, z = 10, 20, 30
```

These flexible assignment techniques are used frequently in real Python programs.

Python Data Types

Python offers a variety of data types. Understanding them is important because each data type has its own characteristics, memory usage, and operations.

Numeric Types

Python supports:

int → Whole numbers

float → Decimal values

complex → Numbers with real & imaginary parts

Examples:

a = 5

b = 3.14

c = 2 + 3j

During training, I used numeric types for mathematical operations, grade calculations, and conditional logic.

String Type

A string is a sequence of characters enclosed in quotes:

name = "Mayank"

Strings support indexing and slicing:

name[0] # 'M'

name[2:5] # 'yank'

We used strings in almost every basic program, such as printing messages, handling user input, and displaying output.

Boolean Type

Booleans have two values:

TrueFalse

They are mostly used inside conditions:

if isOnline:

  print("User is online")

This topic was particularly useful during program development, especially when handling input from users.

Practical Exercises Completed

- Declaring variables of all types
- Writing programs using print statements
- Performing arithmetic operations
- Checking dynamic typing using type()
- Working with strings and slicing

Primary Data Types:

Data Type	Description	Example
int	Whole number	10, -5
float	Decimal Values	10.5, 3.14
str	Text/characters	“Python”
bool	True/False values	True, False

Table 01: Summarized Primary Data Type

Collection Data Types:

Type	Nature	Example
list	Ordered & mutable	[1,2,3]
tuple	Ordered & immutable	(10,20)
set	Unordered, no duplicates	{1,2,3}
dict	Key-value pairs	{“name” : “Mayank”}

Table 02: Summarized Collection Data Type

Chapter 3

Control Statements & Input/Output Operations

Control statements are one of the most important parts of any programming language because they allow the programmer to control the flow of execution. In Python, these statements help the program make decisions, repeat tasks, and perform actions based on user input. During the training, I learned how Python handles decisions using if-elif-else, how looping structures work, and how to interact with users using **input()** and **print()** functions.

Understanding Control Flow

Control flow refers to the order in which Python executes statements in a program. Normally, Python executes code **from top to bottom**, but control statements allow programs to:

1. Make decisions
2. Execute or skip code blocks
3. Repeat tasks
4. Validate conditions
5. Handle user-driven logic

This chapter helped me understand how programs behave based on conditions rather than fixed instructions.

Decision-Making Statements

Decision-making is implemented using:

- if
- elif
- else

These statements check conditions using **relational operators** (>, <, ==, !=, >=, <=) and **logical operators** (and, or, not).

Syntax:

if condition:

Executes when condition is true elif another_condition:

Executes when the above condition is false but this is true else:

Executes when all conditions are false

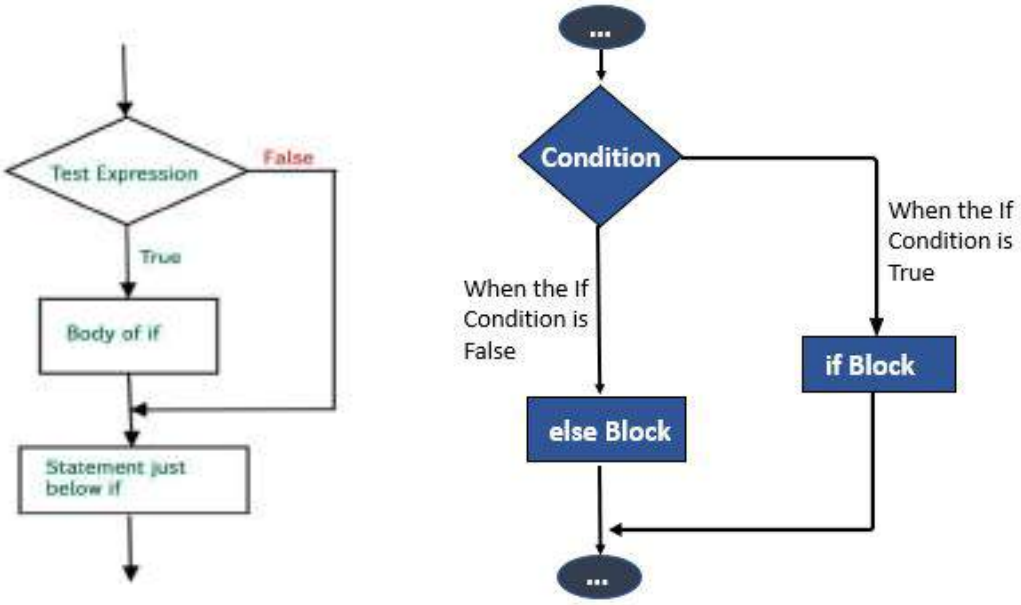


Figure 2: Python If-Else Flow Diagram

This diagram helped me visualize how Python chooses which block to execute based on conditions.

Example 1: Checking Even or Odd

```
num = int(input("Enter a number: "))

if num % 2 == 0:

    print("Even number")else:

    print("Odd number")
```

Example 2: Largest Among Three Numbers

```
a = 10

b = 25

c = 5

if a > b and a > c:

    print("A is largest")elif b > a and b > c:

    print("B is largest")else:

    print("C is largest")
```

This exercise improved my logical thinking during the training.

Relational & Logical Operators

These operators are essential for comparing values:

Operator	Meaning	Example	Result
>	Greater than	5>2	True
<	Less than	3<1	False
==	Equal to	5==5	True
!=	Not equal	7!=2	True
>=	Greater/ equal	5>=4	True
<=	Less/ equal	3<=3	True

Table03: Relational Operators in Python

Logical Operators

and – All conditions must be true

or – Any one condition must be true

not – Reverses the condition

User Input Handling

Python uses the built-in function **input()** to take user input as a **string**.

```
name = input("Enter your name: ")

age = int(input("Enter your age: "))

print("Hello", name)
```

By converting input into int, float, or other types, we can perform numerical operations easily.

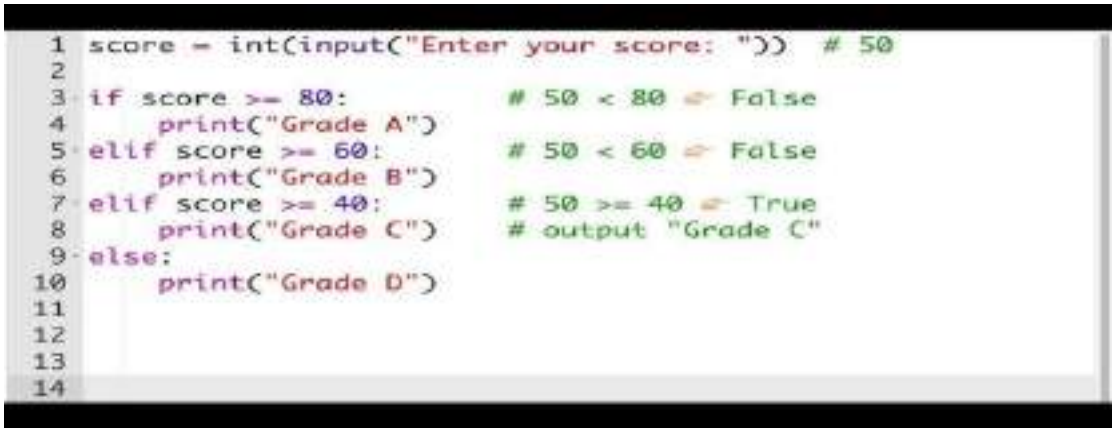


Figure 3: Python Input/Output Console Interaction

This figure shows simple I/O interaction between the program and user.

Conditions with User Input (Practical Tasks)

Task 1: Grade Calculator

```
marks = int(input("Enter marks: "))

if marks >= 90:

    print("Grade A")elif marks >= 75:

    print("Grade B")elif marks >= 60:

    print("Grade C")else:

    print("Grade D")
```

Task 2: Simple Calculator

```
a = int(input("Enter first number: "))

b = int(input("Enter second number: "))

op = input("Enter operator (+, -, *, /): ")

if op == "+":

    print(a + b)elif op == "-":

    print(a - b)elif op == "*":

    print(a * b)elif op == "/":

    print(a / b)else:

    print("Invalid operator")
```

These beginner-friendly exercises helped me understand real-life applications of conditional logic.

Nested If Statements

Python allows placing conditions inside another condition.

num = int(input("Enter number: "))

if num > 0:

print("Positive number")

```
if num % 2 == 0:

    print("Also Even")else:

print("Negative or Zero")
```

Nested logic is useful for multi-level decision making.

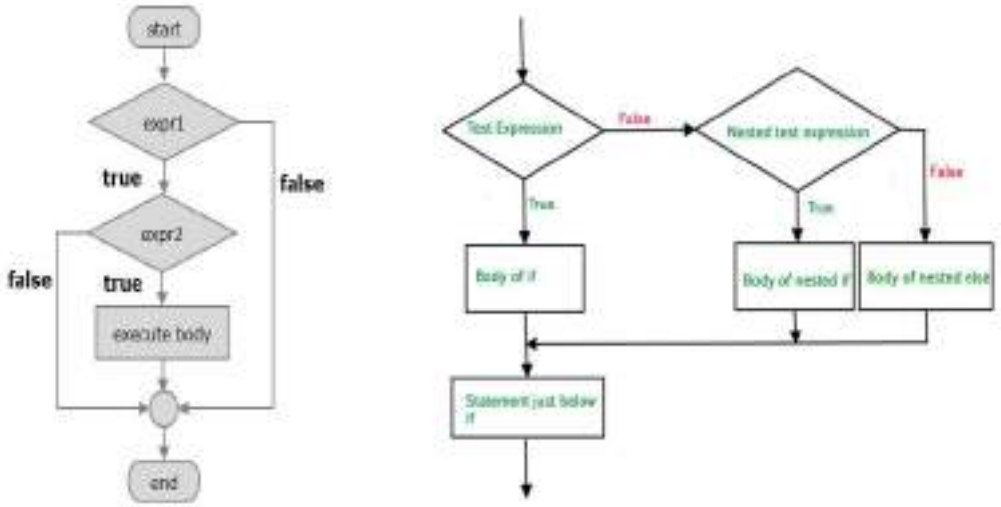


Figure 4: Nested If Decision Structure

This figure helped me understand how nested conditions flow in Python.

Real-World Applications Learned During Training

Some simple real-world programs I practiced:

- Login validation
- Age verification (18+ check)
- Temperature-based advice
- Discount calculation for shopping cart
- ATM-like menu decision flow

Control statements make such programs possible.

These concepts formed the foundation of my problem-solving approach while writing Python programs during the internship.

Chapter 4

Functions & Modular Programming

Today's session introduced me to the concept of **functions** in Python and how they help in writing clean, reusable, and modular code. Understanding functions is essential for building larger programs, reducing redundancy, and improving program structure. This chapter also covered arguments, return values, scope, and the basics of modular programming using Python modules.

Introduction to Functions

A **function** is a block of organized, reusable code designed to perform a specific task. Instead of writing the same code repeatedly, functions allow us to define behavior once and reuse it as needed.

Python provides both **built-in functions** (like `print()`, `len()`, `type()`) and **user-defined functions** created using the `def` keyword.

Syntax of a Function

```
def function_name(parameters):

    # statements

    return value
```

Example

```
def greet():

    print("Hello, welcome to Python!")

Calling the function:

greet()
```

Types of Functions

1. Built-in Functions

Python includes many useful built-in functions such as:

- print()
- len()
- input()
- max() / min()
- range()

These help perform everyday tasks without writing custom logic.

2. User-Defined Functions

Created by the programmer to perform specific tasks.

Example:

```
def add(a, b):
    return a + b
```

Parameters and Arguments

Functions can accept values to work with, called **parameters** or **arguments**.

Positional Arguments

Values passed in order:

```
def multiply(x, y):
    return x * y
```

Default Arguments

Provide fallback values:

```
def info(name="User"):

    print("Hello,", name)
```

Keyword Arguments

Pass values using parameter name:

```
info(name="Mayank")
```

Return Statement

The return keyword sends a value back to the function caller.

Example:

```
def square(n):

    return n * n
```

Returning Multiple Values

Python allows returning multiple values using tuples:

```
def calculator(a, b):

    return a+b, a-b, a*b
```

Variable Scope

Understanding where variables are accessible is important.

1. Local Variables

Defined inside a function.

2. Global Variables

Defined outside and accessible everywhere.

Example:

```
x = 10   # global

def show():

    y = 5  # local

    print(x, y)
```

Modular Programming in Python

Modular programming means splitting code into smaller, manageable files known as **modules**.

Importing Modules

import math

import random

from datetime import datetime

Creating a Custom Module

```
File: myfunctions.py

def welcome():

    return "Hello from module!"
```

```
Using the module:

import myfunctionsprint(myfunctions.welcome())
```

Advantages of Using Functions and Modules

- Improves code readability
- Reduces repetition
- Organizes code into logical sections
- Makes debugging easier
- Allows teamwork and scaling of projects

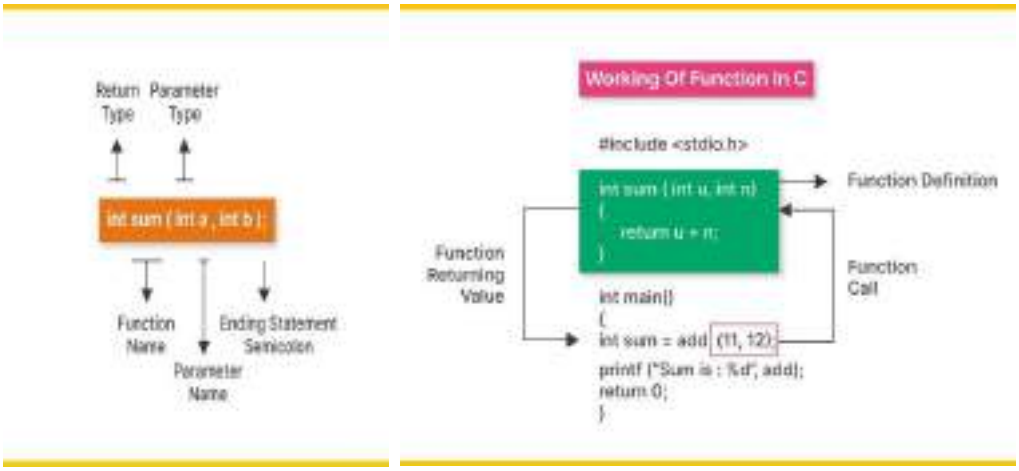


Figure 05: Function Definition and Execution Flow Diagram

This diagram shows the three-step process:

Function call from main program

Execution inside function body

Returning result to caller

Chapter 5

Looping Concept

Loops are one of the most essential parts of any programming language because they allow repetitive execution of code without writing the same statements multiple times. In Python, loops help automate tasks, process lists, read files line-by-line, create patterns, and perform calculations efficiently.

During my training, I learned two primary loop types in Python — **for loop** and **while loop** — along with loop-control keywords including **break**, **continue**, and **pass**. These concepts strengthened my logical thinking and gave me confidence to write basic automation-style programs.

Understanding Iteration

Iteration means repeating a block of code until a specific condition becomes false.
For example, printing numbers from 1 to 10 manually would require 10 print statements, but with loops, this requires only 2–3 lines of code.

Python handles iteration in a simple and readable way compared to many other languages.

For Loop in Python

The **for loop** is mainly used when we know exactly how many times the loop should run. Python's for loop is more powerful because it works with sequences like lists, strings, and ranges.

Syntax

for variable in sequence:

block of code

Example

```
for i in range(1, 6):

print(i)
```

This prints numbers from 1 to 5.

I learned that range() is extremely useful because it can generate sequences based on start, stop, and step values.

While Loop in Python

A **while loop** runs until its given condition becomes false.
It is useful when we do not know in advance how many iterations are needed.

Syntax

while condition:

code block

Example

```
i = 1

while i <= 5:

print(i)

i += 1
```

This loop keeps running until i becomes greater than 5.

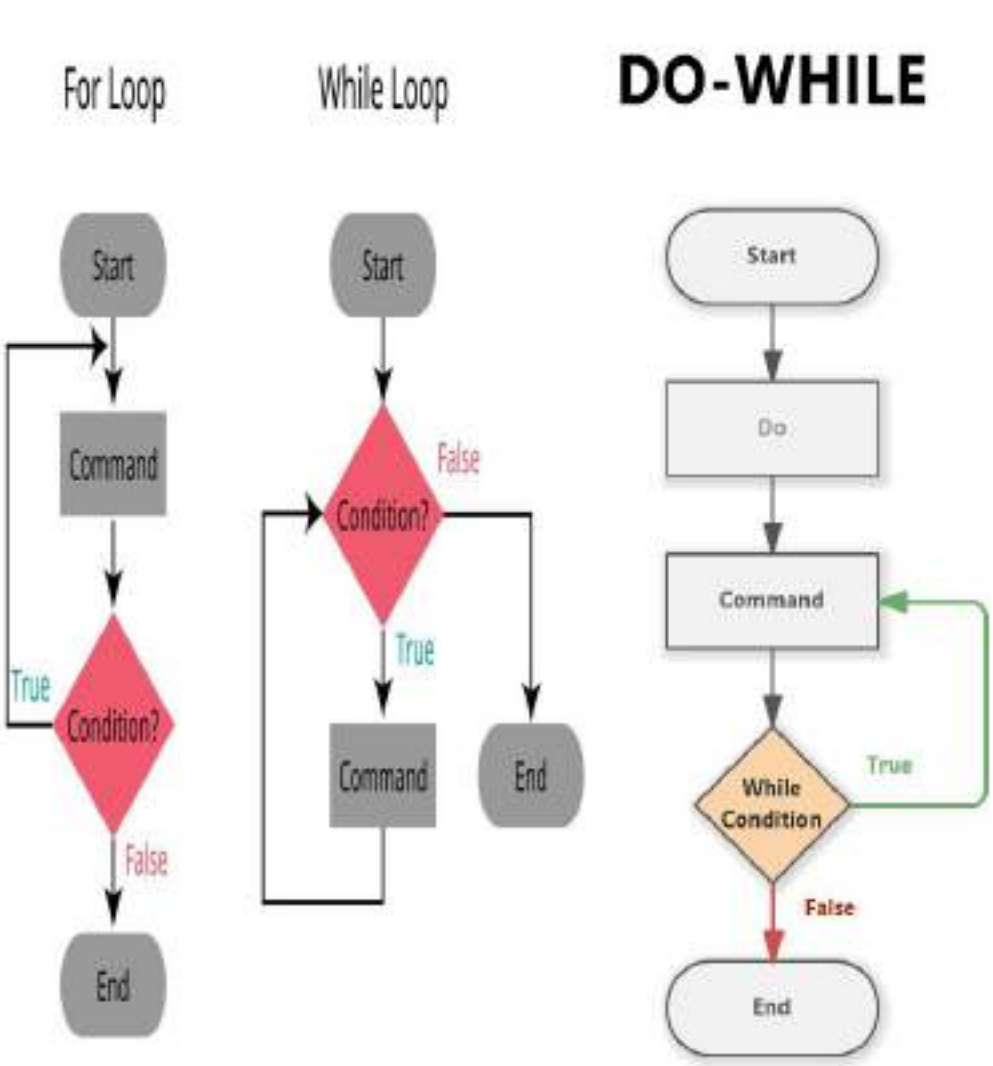


Figure 06:: Loop Flow (For Loop vs While Loop vs Do While Loop)

This diagram helps visualize how both loops operate and how control moves inside the loop.

Loop Control Statements

Python provides three keywords to control loop execution:

1. break - Stops the loop immediately.

2. continue- Skips the current iteration and moves to the next one.

3. pass- Does nothing — used as a placeholder when the code is not ready**.**

Table: Difference Between For Loop and While Loop

Feature	For Loop	While Loop
Use Case	When number of iterations is known	When number of iterations is unknown
Based On	Sequence or range	Condition
Risk	Less chance of infinite loop	Higher chance if condition not handled
Ease of Use	Simple and clean syntax	Requires manual update of variable

Table 5: For Loop vs While Loop

This table helped me understand when to choose which type of loop in a program.

Practical Programs Covered in Training

A. Printing Even Numbers

```
for i in range(1, 21):

if i % 2 == 0:

print(i)
```

B. Sum of First 10 Natural Numbers

```
total = 0

i = 1

while i <= 10:

total += i

i += 1
```

```
print(total)
```

C. Simple Number Pattern

** ** * ** * * *

```
for i in range(1, 5):

print("* " * i)
```

Key Learnings from This Chapter

- Loops reduce repetition and make programs efficient.
- for loops are ideal for fixed-length iterations.
- while loops are ideal for condition-based repetition.
- Loop control keywords improve flexibility and flow-control in programs.
- Visualizing loop flow improves understanding of program execution.

Chapter 6

Python Data Structures: Lists, Tuples, Sets, and Dictionaries

Data structures are one of the most important parts of Python programming. During the training, I learned how Python provides built-in data structures that help store, organize, and process data efficiently. These data structures are simple to understand and use, making Python extremely beginner-friendly.

Python's four most commonly used data structures are:

1. **List**
2. **Tuple**
3. **Set**
4. **Dictionary**

Each of them works differently and is used for different purposes. In this chapter, I explain all four in detail based on what I learned during my internship.

1. Lists

A **List** in Python is a dynamic, ordered collection of items. It is one of the most frequently used structures because it can store multiple items in a single variable.

Key Features of Lists

- **Ordered** (items maintain their index)
- **Mutable** (we can modify elements)
- **Allows duplicates**
- Can store **different data types** (e.g., integers, strings, floats together)

Syntax

```
my_list = [10, "Python", 3.14, True]
```

Common List Methods Learned

append() – adds an element

remove() – removes an element

sort() – sorts the list

pop() – removes last element

insert() – inserts at a specific index

Example

```
numbers = [5, 2, 8, 1]

numbers.append(10)

print(numbers)
```

2. Tuples

A **Tuple** is similar to a list but **immutable**, meaning its elements cannot be changed after creation.

Key Features of Tuples

- Ordered
- Immutable
- Faster than lists
- Useful for fixed data

Syntax

```
my_tuple = (10, 20, 30)
```

Example

```
coordinates = (12.5, 5.6)

print(coordinates)
```

Tuples are commonly used when we want data to remain constant throughout the program.

3. Sets

A **Set** is an unordered collection of unique items. It is mainly used to remove duplicates or perform mathematical operations like union, intersection, etc.

Key Features of Sets

- Unordered
- No duplicate elements
- Mutable (but elements must be immutable like numbers or strings)

Syntax

```
my_set = {1, 2, 3, 3, 4}

print(my_set)                    # Output: {1, 2, 3, 4}
```

Useful Set Functions

```
add()

remove()

union()

intersection()
```

4. Dictionaries

A **Dictionary** stores data in **key-value pairs**. It is one of the most powerful data structures in Python and used widely in real-world applications.

Key Features of Dictionaries

- Unordered
- Stores data in key:value format
- Keys must be unique
- Mutable

Syntax

```
student = {"name": "Mayank", "age": 19, "course": "Python"}
```

Accessing Values

```
print(student["name"])
```

Updating Dictionary

```
student["age"] = 20
```

Dictionaries are useful for storing structured data like student profiles, product details.

Collection VS Features	Mutable	Ordered	Indexing	Duplicate Data
List	✓	✓	✓	✓
Tuple	-	✓	✓	✓
Set	✓	-	-	-
Dictionary	✓	✓	✓	-

Table 6 :Features of Python Data Structure

This diagram represents the basic differences I learned during the training.

Chapter 7

String Handling & Built-in Functions in Python

Strings are one of the most essential data types in Python and are used extensively in almost every program. Python treats strings as **ordered, immutable sequences of characters**, which means each character has an index and the string cannot be changed directly after it is created.

To make this chapter visually clear, one diagram is added below.

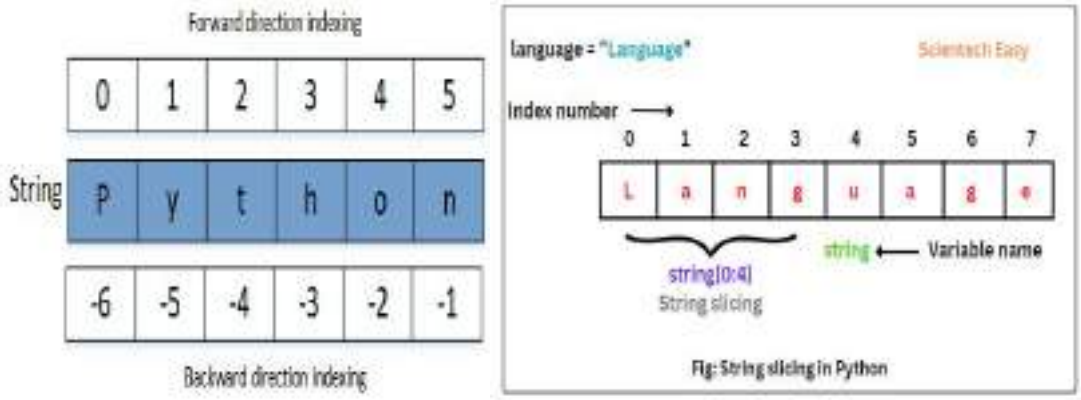


Figure 7: Strings in Python

Introduction to Strings

A string in Python is created using single quotes ' ', double quotes " ", or triple quotes ''' ''' / """ """ for multi-line text.

Examples:

```
name = "Mayank"

message = 'Hello Python!'

para = """This is a multi-line string."""
```

Key Properties

- Strings are **immutable**
- They support **indexing**
- They support **slicing**
- Many built-in functions exist for easy manipulation
- Widely used for input/output formatting, file handling, APIs, and text processing

String Indexing

Python assigns an index to every character of a string.

Example:

M a y a n k

0 1 2 3 4 5 (positive indexing)

-6 -5 -4 -3 -2 -1 (negative indexing)

To access elements:

```
name = "Mayank"

print(name[0])                # M

print(name[-1])               # k
```

String Slicing

Slicing allows extracting a portion of a string.

Syntax:

```
string[start : end : step]
```

Examples:

```
s = "PythonProgramming"

print(s[0:6])      # Python

print(s[6:])       # Programming

print(s[::-1])     # reverse string
```

Common String Functions

Python provides many built-in methods for working with strings.

Function	Description	Example
lower()	Converts to lowercase	“HELLO”.lower()
upper()	Converts to uppercase	“hello”.upper()
Title()	Capitalizes each word	“python basics”.title()
Strip()	Removes leading/trailing spaces	“hello”.strip()
Replace()	Replace part of text	"hello".replace("h","m")
Find()	Returns index of substring	“apple”.find(“p”)

Table 7 : Built-in metods for strings in Python

String Formatting Methods

f-strings (recommended)

name = "Mayank"print(f"Hello, {name}")

format() method

print("My name is {}".format("Mayank"))

Percent (%) formatting

(Older method)

print("Age: %d" % 20)

Escape Characters

Escape characters allow inserting special symbols inside a string.

Escape Code	Meaning
\n	New line
\t	Tab space
\\	Backslash
\”	Double quote
\’	Single quote

Table 8 : Escape characters in Python

Example:

print("Hello\nPython")

Useful String Operations

Concatenation

a = "Python"

b = "Training"print(a + " " + b)

Repetition

print("Hi! " * 3)

Membership Operators

"Py" in "Python" # True"x" not in "text" # True

String Immutability

Strings cannot be changed directly:

s = "Hello"# s[0] = "J" ✗ invalid

Instead, a new string is created:

s = "J" + s[1:]print(s) # Jello

Practical Applications of Strings

- User input validation
- File names and paths
- API responses (JSON strings)
- Authentication systems
- Text analytics & data cleaning
- Creating formatted output for reports

Chapter 8

Exception Handling

During the training, I only got a brief introduction to **exception handling in Python**, but it helped me understand why errors occur and how programs can run more safely by handling them properly.

What is an Exception?

An exception is an **unexpected error** that stops a program while it is running.
Example: dividing a number by zero or accessing an index that does not exist.

Why Exception Handling is Needed?

- To **avoid program crashes**
- To **show meaningful messages** instead of Python errors
- To **continue execution safely**
- To make programs **more user-friendly**

Basic Try–Except Block

I learned the simplest form of exception handling:

```
try:

  a = 10 / 0    except Zero Division

  Error:

    print("Division by zero is not allowed")
```

Finally Block

I only learned that finally is used to run code **always**, even if an error occurs.

```
try:
    file = open("test.txt")finally:
    print("Program ended")
```

Common Built-In Exceptions

- **ZeroDivisionError** – when dividing by zero
- **ValueError** – wrong type of input
- **TypeError** – incorrect data type operations
- **IndexError** – index out of range

I didn't explore advanced exceptions due to time limitations.

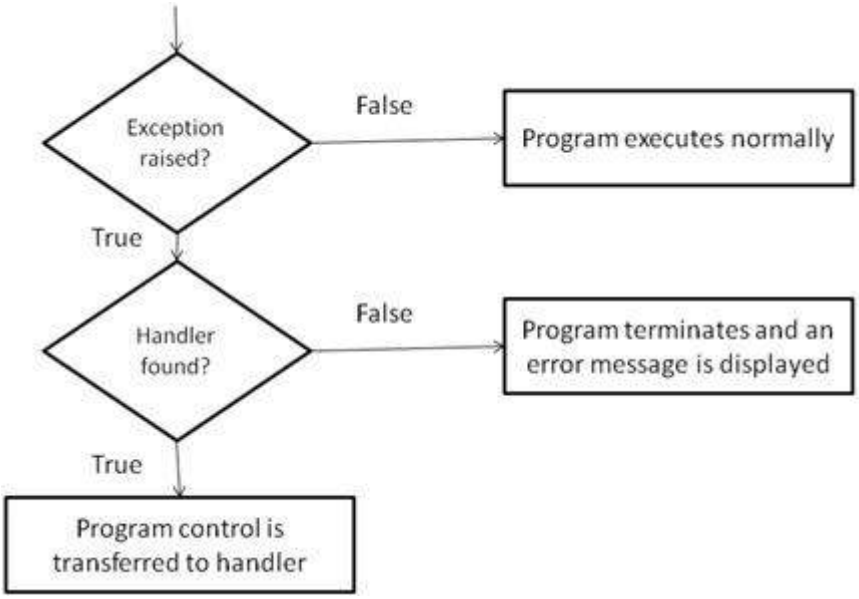


Figure 08: Basic Try–Except Flow Diagram

BOLO.COM – Voice-Enabled AI Chatbot

BOLO.COM is a multilingual voice-based AI chatbot developed using Flask, JavaScript, and OpenAI GPT models. The system allows users to interact using spoken Hindi or English, and the chatbot replies in the same language, both as text and speech. This project demonstrates practical integration of speech technologies with AI-powered text generation.

Objective

- To build a real-time voice-interactive chatbot.
- To support Hindi and English conversations using auto language detection.
- To integrate Speech-to-Text (STT), AI response generation, and Text-to-Speech (TTS) into a single web application.
- To create a simple and responsive user interface for easy interaction.

Technologies Used

Frontend

- HTML, CSS, JavaScript
- Web Speech API (SpeechRecognition + SpeechSynthesis)

Backend

- Python, Flask, Flask-CORS
- OpenAI API (GPT-4o-mini model)
- VS Code
- Virtual Environment (.venv)

System Overview

The application follows a client–server model:

Client Side (Browser)

1. Listens to the user’s voice and converts it into text using the Web Speech API.
2. Sends the text to the Flask backend through a POST request.
3. Receives the AI-generated reply and displays it in the chat interface.
4. Uses Text-to-Speech to read the reply aloud.

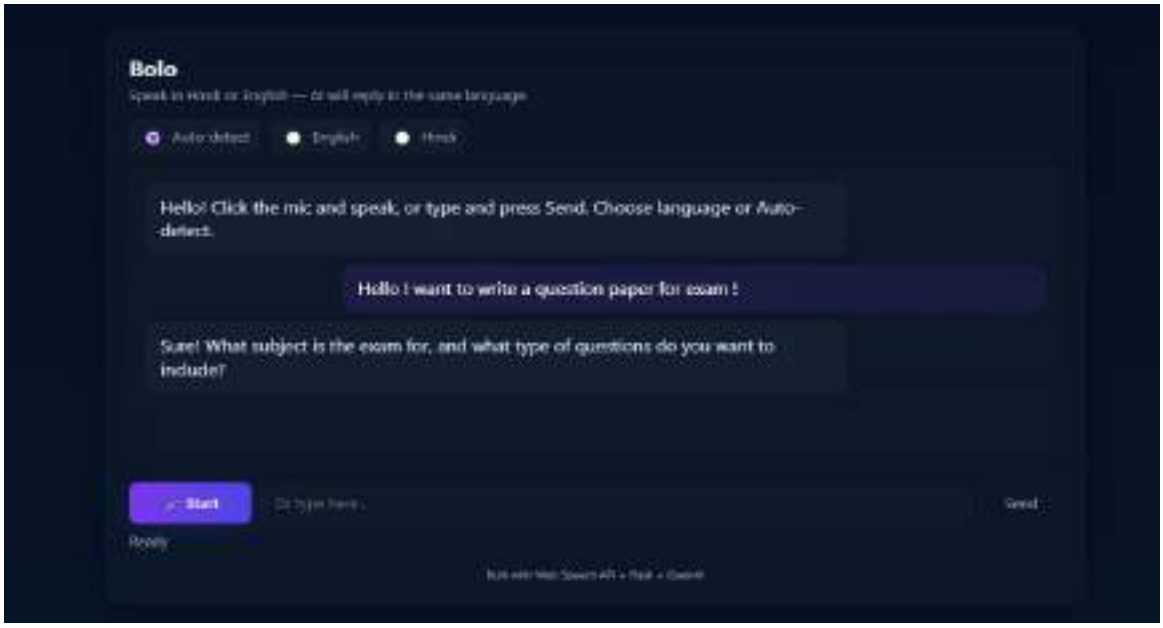
Server Side (Flask Backend)

5. Receives the user's message and language preference.
6. Sends the message to the OpenAI GPT model with a system prompt.
7. Processes the response and sends it back as JSON.

Figure 09 : Workflow of Bolo

Key Features

- Multilingual Support – understands and replies in Hindi and English.
- Speech-to-Text Input – hands-free interaction.
- AI-Generated Responses – powered by GPT-4o-mini.
- Text-to-Speech Output – chatbot speaks replies.
- Clean UI – chat-style interface with message history.



UI of Bolo

Applications

- Customer service assistants
- Voice-enabled websites
- Education and e-learning bots
- Accessibility tools
- AI research projects

Conclusion

BOLO.COM demonstrates how modern AI and web technologies can be combined to build an interactive, multilingual voice assistant. The project strengthened skills in Flask backend development, API integration, speech processing, and AI-driven user interfaces. Overall, it serves as a practical example of real-time AI communication in web applications.

Training Summary

The 15-day industrial training on Python Programming provided me with a clear and practical introduction to the fundamentals of Python and its applications. The program was designed to offer a beginner-friendly, hands-on learning experience, allowing me to gradually understand how Python works and how it can be used to solve real-world problems.

Throughout the training, I learned essential concepts such as Python syntax, variables, data types, operators, conditional statements, loops, functions, and basic file handling. These foundational topics helped me develop logical thinking and understand how to write simple and structured programs. The sessions also included multiple small exercises, which strengthened my understanding of flow control, user input handling, and modular programming.

I was introduced to Python's built-in data structures like lists, tuples, sets, and dictionaries, which gave me insight into how data can be stored and processed efficiently. Along with this, I gained basic exposure to exception handling and modules, although these topics were covered only at an introductory level due to the short duration of the internship.

The training followed a practical-oriented approach, where each topic included small examples and practice problems. This helped me apply the concepts immediately and understand Python's readability, dynamic typing, and flexibility. The inclusion of diagrams, tables, and a few simple programs made the learning process engaging and easy to follow.

Overall, this internship helped me build a strong foundation in Python programming. Even with limited time, I gained enough understanding to write basic programs and explore Python further on my own. The training has motivated me to continue learning advanced topics like object-oriented programming, libraries, and automation in the future.

Key Outcomes

- Gained a clear understanding of Python fundamentals, including variables, data types, operators, and basic syntax.
- Learned to apply control statements, loops, and functions to build simple, logical programs.
- Understood and practiced essential Python data structures such as lists, tuples, sets, and dictionaries.
- Developed familiarity with string handling, basic file operations, and simple error handling.
- Completed small exercises and a mini project that improved problem-solving ability and confidence in writing clean Python code.

Next Steps

- Strengthen core concepts by solving more Python practice problems.
- Explore advanced Python areas such as modules, libraries, and automation.
- Begin learning beginner-friendly domains like data analysis (NumPy, pandas) or web development using Flask/Django.
- Improve code quality through version control tools like Git and regular coding practice.

References

- Python Official Documentation

  https://docs.python.org/3/

- W3Schools – Python Tutorial

  https://www.w3schools.com/python/

- GeeksforGeeks – Python Programming

  https://www.geeksforgeeks.org/python-programming-language/

- Real Python – Python Guides & Tutorials

  https://realpython.com/

- Programiz – Python Basics

  https://www.programiz.com/python-programming

- TutorialsPoint – Python Programming Language

  https://www.tutorialspoint.com/python/index.htm

- Stack Overflow – Developer Community Discussions

  https://stackoverflow.com/